Ant Colony Optimization for the 0/1 Knapsack Problem

Student id: 740036129
University of Exeter
ECMM409 ACO

Abstract

This paper focuses on generating a near-optimal and optimal solution for the 0/1 knapsack problem for the provided $BankProblem.txt$ dataset by adjusting the Ant Colony Optimization (ACO). Here we try to maximise the tonal value of all bags collected without going over 295kg weight limit for the van.

1 Introduction

In this paper we explore the use of ant colony optimisation (ACO) for the approximation and finding of near to optima solutions for the 0/1 Knapsack Problem for the *BankProblem.txt* dataset what has a maximum capacity of 295kg.

1.1 The 0/1 Knapsack Problem

In this problem we have a set of bags, each bag has a *id*, *wight* and a *value*. The aim for this problem is to fill the van with as much value as possible, however the van has a weight limit of 295kg and the the total weight of all the bags in the dataset is 539.9kg as such we need to select the best possible bags to load the van with.

The *BankProblem.txt* dataset contains 100 bags. The total value of all of the bags is £5628 and when searching purely randomly it falls around £3000 to fitting into the van.

2 Literature Review

This entire section has two major references one being (Hristakeva and Shrestha(2005)) and (bra(2024)). The paper covers everything, the geeksforgeeks goes into detail for most covered but has a special focus on Branch and Bound 2.3.

2.1 Greedy Algorithm

The greedy algorithm is a quick way of generating an estimate, that would compute a **value-to-weight** ratio (v/w) for each bag, this algorithm then sorts the highest to lowest and starts searching from there. This means that the higher ratio values are guaranteed to make it, but as it gets to lower values it will skip over bags that potentially in combination with other bags would have a much better ratio.

Overall this is a quick way of gathering an estimate and would perform way better than random due to having a very good heuristic for this problem. However would suffer from missing out on potential smaller bag combinations meaning it would potentially not reach the optima.

2.2 Genetic Algorithm

One potential approach for solving this would be to use a **Genetic Algorithm** (GA). In the GA, it would generate a population, in the each individual contains *chromosomes*, each one representing a individual bag (True or False). This through the selection, crossover and mutation process (along side repair what would be needed if a individual violates the weight limit), would progressively find better solutions till it either falls into a local optima or find the optima solution.

2.3 Branch and Bound

This approach builds upon the Greedy Algorithm mentioned earlier. This works by breaking down the problem and placing it into a tree, this then will using the greedy algorithm, determine weather or not each branch is worth while in a bottom up approach following a depth first search behaviour pattern.

This algorithm generates the actual maximum value, through searching and ensuring that it does not discard a branch until it is fully explored - only discarding if it does not fulfil the weight requirements. Running our dataset through the *geeksforgeeks.org* implementation we find that the max-

imum possible value placed in the van for the *BankProblem.txt* dataset is £4528. ((bra(2024)))

2.4 Brute Force

This method similar to Branch and Bound will find the best possible solution, in a much simpler way, however is dramatically more computationally expensive as it searches all possible solutions. As such as a big-o notation of $O(n) = 2^n$. So even if we go for 10,000 checks, this algorithm will still be searching. ($O(100) = 2^100$) (Hristakeva and Shrestha(2005)).

3 Methodology

Algorithm Design

The provided $BankProblem.txt$ dataset consists of the total van capacity and a set of 100 bags where each bag has a *bag number*, *weight* and *value*. Building on this, the following section addresses the decisions made regarding adjustments to the default ACO algorithm to achieve results ranging between £4527 and £4528.

This sections starts over a very important aspect being the heuristic, covering why it was chosen. Next we cover the pheromone matrix, going over the fitness function and how this is then used to determine how much each deposit is placed onto this matrix.

3.1 The heuristic

A good heuristic will guide the algorithm in the correct direction.

As previously stated we are given a dataset containing 100 values and weights for each bag; the heuristic values need to each be a single value - to solve this problem value is divided by weight for each bag:

$$vpw = \frac{value}{weight}$$

Value Per Weight (vpw) is now an array size $(100,)$, after taking the maximum vpw_{max} and minimum vpw_{min} measurements this can be then taken and normalised the data between 0.0 and 1.0 to produce a very easy metric to work with.

$$vpw_i = \frac{vpw_i - vpw_{min}}{vpw_{max} - vpw_{min}}$$

As such vpw can now be seen that the value's in the set are ranging between 0.0, being the worst and 1.0 being the best. It is important to note bags can still be explored due to randomness in the initial pheromone matrix, but these low values highly not recommended by the heuristic.

After this vpw is used to generate the vertical columns for the heuristic matrix, ultimately unlike the travailing salesmen problem there are no distinct distances between bags, so the next bag has no direct distance relation to the previous bag picked and the current but what the will be found by the ACO algorithm. This is purely due to the structure of the *BankProblem* dataset that these relations are made.

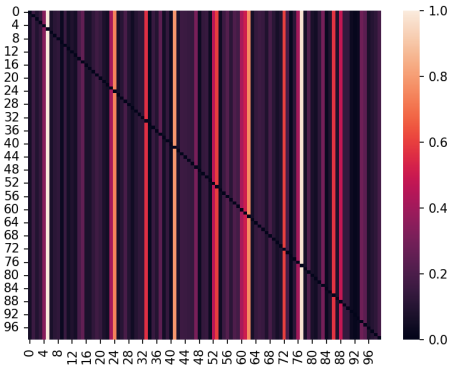

Figure 1: Illustration of the Heuristic Matrix

The horizontal can be seen to be full of zeros, this is because we do not want the ant to risk re-visiting itself.

Ultimately this heuristic was chosen because it effectively directs the search process toward optimal solutions, eliminating the need for the ACO algorithm to perform additional searches (and risking settling in non-optimal solutions), which would increase the computational load needed to find near-optimal solutions in this case.

3.2 The pheromone matrix

The initial pheromone matrix is just a $(100, 100)$ with values between 0.0 and 1.0 with full zeros across the horizontal.

After the pheromone matrix has been well established by going through enough generations patterns do appear in regards to where the ant prefers to visit. Interestingly due to the structure of the

Heuristic matrix the vertical columns become partially viable across some node visitation patterns.

3.2.1 Pheromone distribution and the fitness function

Here we use 10 ants as such there are solutions S_N represents the sum of all values in a solution, where $N \in \{1, 2, 3...., 10\}$

Using inspiration from elitist evolutionary algorithms, a basic fitness mask was created where the worst 20% of values are removed

$$S_N = \begin{cases} 0 & \text{if } S_n \leq S_{worst} + 0.2 \cdot (S_{best} - S_{worst}) \\ S_n & \text{otherwise} \end{cases}$$

Note: to worst and best values are precomputed before applied to the set so no skew is made to the resultant solution value set.

The fitness is then proportional to those fitness values.

Here we already get pretty good outputs from the ACO algorithm (given good adjustments made to τ and η, α and β). If the heuristic was much worse and didn't already make the ACO algorithm find such good solutions an alternative fitness function could have been implemented.

Elitism can be used here to squee the solutions towards finding better solutions, here we settled on a basic 20% mask that would set the worst solutions to 0 - thus removing any pheromone from being able to be deposited. This overall tipped the solutions to reaching much higher values.

Reducing this threshold for 10 ants, it was found that this did not make an optimal impact on the outcome (i.e., 10% or less) and where the threshold was much higher it was found that yes at first it would continue to get better but when it reached a certain margin, this started to impact the ACO's ability to search. As such the 20% was settled on as it aided the search process.

An Alternative Approach that could be taken: Alternatively if we were not already getting such good results another fitness function might perform better. The suggestion would be to look towards using a sigmoid function on the values, skew that slightly towards better solutions then use that after normalising the values between 0 and 1, then standardise that for all values so it becomes a sum total of τ to determine pheromone deposit for a set of solutions.

3.2.2 Evaporation

An important thing to note, even though its not been talked about directly here, each set once given its pheromone deposit further breaks that down to evenly distribute between all values for each deposit for every node. As such an important observation can be made that the impact of a single ant visiting one node is small, but the cumulative nature of the ants pheromone creates a major impact - as such evaporation might take away what most if not all of what one ant deposits but with enough ants visiting any given node it makes it will become a trail they often follow.

Here we only use about a 5% evaporation rate ($0.95 \cdot$ pheromone matrix).

4 Results, Discussion and Further Work

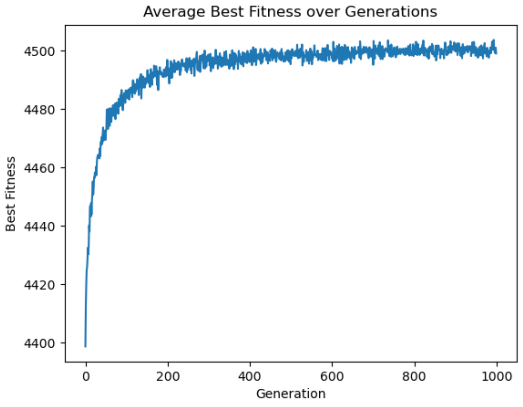

Figure 2: Illustration showing the average best over each generation of ants given that there are 10 ants per generation, 1000 generations and 10,000 total fitness evaluations.

4526	4527	4528	Total
1	361	138	500
0.2%	27.6%	72.2%	100%

Table 1: This table shows output occurrences out of 500 samples of running the ACO algorithm

The best solution is £4528 and weighs 294.7kg can be seen in 3. The average solution is £4527.72

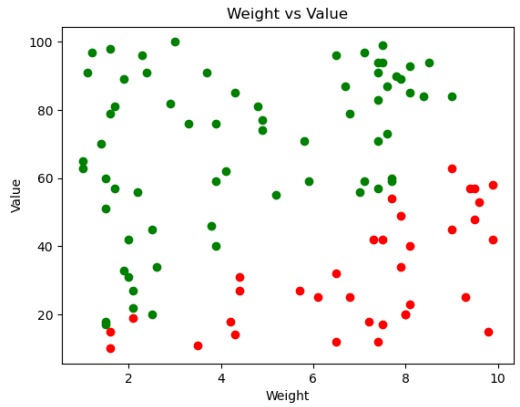

Figure 3: The Best Solution £4528 and weighs 294.7kg. These bags in this solution are always: 2, 3, 4, 5, 6, 8, 9, 11, 15, 16, 19, 20, 21, 24, 25, 28, 29, 31, 32, 33, 34, 36, 37, 38, 40, 42, 44, 45, 46, 47, 48, 50, 51, 53, 54, 56, 57, 58, 59, 60, 61, 62, 63, 65, 66, 67, 69, 72, 73, 74, 76, 77, 78, 80, 83, 84, 87, 89, 90, 91, 95, 96, 98, 99, 100

(cannot achieve this value but more than not its optimal).

4.1 Which combination of parameters produces the best results?

The best results were found using these values:

- τ - 1.0 Amount Deposited Each Generation ($\sum$(deposited on every node))
- τ_{max} - 1.0 A cap stopping a node being over-saturated. thus keeping it proportional to α and β
- α - 1.0 Proportional impact of pheromone τ on search
- β - 2.0 Proportional impact of heuristic on search
- p - 0.95 evaporation rate (only 5% removed)
- Population - 10

This was all tested on 10000 evaluations, where each evaluation is one set (S_N) of bags up to 295kg. Thus 10 ants meaning there were 1000 generations.

There are improvements that are possible but that is only marginal between runs by £1 when it already finds the best solution most of the time (£4527 vs. £4528).

4.1.1 Why are these results produced and why do these parameters influence in such a way? (Q1-3)

These results are produced due to a combination of the ACO algorithm searching and there being a very good heuristic for the dataset.

Ultimately this is due to the dataset being so simple with such a limited way that it can be adapted for creating a heuristic matrix as seen in section 3.1. Based on this good heuristic, the ants quickly capture the relationship, additionally the use of the value based fitness function masking out poor solutions speeding up the convergence towards a good search space. These ants then have a good area to explore and revisit near optimum solutions often.

The 2 parts β keep the solution looking towards very good bags to select and the α allows the algorithm to search around this for better combinations of solutions - this can be highlighted very well in figure 1.

Given α and β using such a low amount evaporated each generation in combination with such a low amount of pheromone deposited (when looking at a single bag for a single ant visit), it means that when it goes to a poor solution if not caught out by the fitness function such a low amount of pheromone will not create major changes. However when it starts to find a better path it really latches on over a couple generations as it is cumulative even with parts being evaporated slowly. This explains why it finds £4528 more so than it does £4527 as the final solution and can even be seen in figure 2.

The use of a maximum τ_{max} value has had a very large impact in improving reliability of producing these results, without τ_{max} resultant values would range between 4519 to 4528 rather than the 4527 and 4528 that we see. This value just means that it takes over a part of the role of the p evaporation but not fully! Evaporation still plays a major role in keeping the pheromone from building up on non-optimal moves to specific nodes (bags).

A key point with figure 2 the best fitness is an average for a single generation of 10 ants it often finds £4527 or very close give or take 1) before

the 200^{th} generation, even saying that it often happens around 100^{th}. This figure is an average and shows the search space slowly narrowing as it goes to higher values, one bag swap or a set of small bag swaps would account of this being lower by £28 or more on average), this does not mean however its not visiting these high values often (as it does and is).

Keypoint: A very easy way of summarising what is going on is that the 2 parts (β) are keeping the algorithm searching a good area but on its own can only find the best solution through brute force (possible with 10,000 attempts but improbable), the ACO algorithm given this heuristic β, takes over and lowers the search area with its 1 part (α), progressively narrowing it down till it lands more and more constantly on the near-optimal and optimal solutions, and this starts working very quickly. Importantly there is always a chance it will not fall on the optima solution but slightly off, and this is due to randomness (ACO is only supposed to produce really good approximations, its just a matter of this being a simple dataset and a good heuristic we can get so close if not find it).

4.2 Does this ACO algorithm perform better than any of the alternative approaches? (Q4)

This ACO algorithm will likely perform better than two of them but on par with the other two when it comes to value (ranked in approximate order of worst to best):

- **Brute Force:** Despite knowing this will produce the actual optima, this takes way longer to perform as such the ACO algorithm will likely have enough time to be run multiple times - giving it enough time to find the optima if not found in the first run (even with 10,000 evaluations per run).
- **The Greedy Algorithm:** This will not perform as well as the ACO algorithm due to previously discussed problems in the literature review - this is not capable of multiple small bags vs one big bag comparison as such will miss these optimisations.
- **A Standard Genetic Algorithm:** This will likely not perform as good as the dataset contains 100 bags, these bags will likely create some local optima that it can fall into as such this algorithm can only be assumed to either perform on-par or worse than this ACO without testing.
- **Branch and Bound:** This will find the optimal solution as such will outperform the ACO algorithm and can be said to perform faster as well doing to testing with an alternate implementation (bra(2024)).

  Branch and Bound outperforms ACO in value 27.8% of the time with the remaining 72.2% of the time achiveing the same result.

4.3 Future Work

Continuing on from this there is one area, I have felt that I myself have not explored and that is alternate fitness functions such as using a sigmoid function. As such this would be the next area to explore for ACO.

References

2024. 01 Knapsack using Branch and Bound. https://www.geeksforgeeks.org/0-1-knapsack-using-branch-and-bound/

Maya Hristakeva and Dipti Shrestha. 2005. *Different Approaches to Solve the 0/1 Knapsack Problem.* https://micsymposium.org/mics_2005/papers/paper102.pdf Incredibly useful for all approaches looked into in literature review..